Strategy Pattern

Category: Behavioural Design Pattern

1. The Problem

Consider a payment system where users can pay using different methods — credit card, debit card, UPI, etc. Without the Strategy Pattern, this is often handled with `if-else` / `switch` conditions inside a single service class.

```
class PaymentServiceIn {

    public void processPayment(String paymentMethod){
        if(paymentMethod.equals("Credit Card")){
            System.out.println("Making payment via credit card");
        } else if (paymentMethod.equals("Debit Card")) {
            System.out.println("Making payment via debit card");
        } else {
            System.out.println("Unsupported Payment Method");
        }
    }
}

public class WithoutStrategyPattern {
    public static void main(String[] args) {
        PaymentServiceIn paymentService = new PaymentServiceIn();
        paymentService.processPayment("Debit Card");
        paymentService.processPayment("Credit Card");
    }
}
```

Issues: - `PaymentService` has multiple responsibilities — it both decides the payment type *and* processes it. - Adding a new payment method means modifying `PaymentService` directly. - Growing `if-else` chains become harder to read and maintain. - Violates the **Open/Closed Principle** — the class is not closed for modification when a new algorithm/payment type is introduced.

2. The Solution — Strategy Pattern

Intent: Decouple the algorithm implementation from the client that uses it, so different algorithms (strategies) can be swapped in and out at runtime without altering the client's code.

- Each payment type's logic is encapsulated in its own **strategy class**.
- The `PaymentService` (the **context**) simply delegates the actual work to whichever strategy object it currently holds.

3. Key Idea

- **Program to an interface, not an implementation** — the context only knows about a `Strategy` interface, never the concrete algorithm classes.

- **Composition over conditionals** — instead of branching logic (`if-else` / `switch`) inside the context, the behavior is *swapped in* as an object at runtime.
- This directly follows the **Open/Closed Principle**: new strategies can be added by creating new classes, with zero changes to the context class.

4. Variants

Not really applicable here — Strategy is fairly consistent in implementation across sources. The main variation seen in practice is whether the strategy is: - **Set explicitly** by the client (`setPaymentStrategy()` , as in this example), or - **Injected via constructor** at object-creation time.

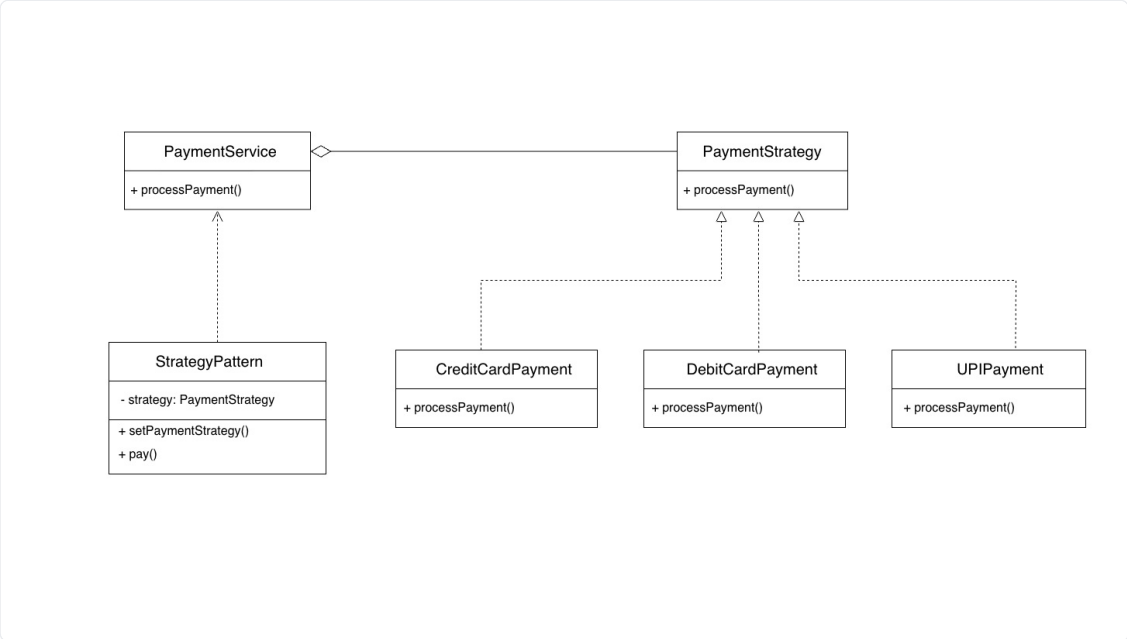
Both are the same pattern; just different ways of wiring the strategy into the context.

5. Structure / Participants

Role	Responsibility	Example (from this exercise)
Context	The client class that holds a reference to a strategy and delegates work to it	`PaymentService`
Strategy (interface)	Declares the operation all concrete strategies must implement	`PaymentStrategy`
Concrete Strategy	Implements the actual algorithm; interchangeable at runtime	`CreditCard` , `DebitCard` , `UPI`

Relationship type: Composition — the `Context` *holds* a reference to a `Strategy` object (the diamond in the diagram points from `PaymentService` to `PaymentStrategy`).

Diagram



Strategy Pattern UML diagram

`PaymentService` (context) composes a `PaymentStrategy` and exposes `processPayment()`. `StrategyPattern` (the client/driver) holds a `PaymentService`, calls `setPaymentStrategy()` to inject a chosen strategy, then calls `pay()`. `PaymentStrategy` is the strategy interface, implemented by the concrete strategies `CreditCardPayment`, `DebitCardPayment`, and `UPIPayment` — each free to define `processPayment()` however it needs to.

6. Code — Full Implementation

```
interface PaymentStrategy {
    void processPayment();
}

class DebitCard implements PaymentStrategy {
    @Override
    public void processPayment() {
        System.out.println("Making Payment via DebitCard.");
    }
}

class CreditCard implements PaymentStrategy {
    @Override
    public void processPayment() {
        System.out.println("Making Payment via CreditCard.");
    }
}

class UPI implements PaymentStrategy {
    @Override
    public void processPayment() {
        System.out.println("Making Payment via UPI.");
    }
}

class PaymentService {
    PaymentStrategy strategy;

    public void setPaymentStrategy(PaymentStrategy strategy){
        this.strategy = strategy;
    }

    public void pay(){
        strategy.processPayment();
    }
}

public class StrategyPattern {
    public static void main(String[] args) {
        PaymentService paymentService = new PaymentService();

        paymentService.setPaymentStrategy(new DebitCard());
        paymentService.pay();

        paymentService.setPaymentStrategy(new UPI());
        paymentService.pay();
    }
```

```
        }
```

What changed vs. the "without pattern" version: - Single `processPayment(String)` method with `if-else` → `PaymentStrategy` interface with one `processPayment()` method, implemented separately per payment type. - `PaymentServiceIn` handled deciding *and* executing payment logic → `PaymentService` now only delegates to whatever strategy object it's holding. - Adding a new payment type used to mean editing `PaymentServiceIn` → now it just means adding a new class that implements `PaymentStrategy`; `PaymentService` is untouched. - Payment method selection moved from a string comparison at call-time to setting a strategy object (`setPaymentStrategy()`) before calling `pay()`.

7. Benefits

- **Open/Closed Principle** — new algorithms/strategies can be added without modifying the context class.
- **Eliminates conditional complexity** — no growing `if-else` / `switch` chains in the client.
- **Single Responsibility** — each strategy class is responsible for exactly one algorithm; the context is only responsible for delegation.
- **Runtime flexibility** — the strategy used by the context can be changed dynamically (`setPaymentStrategy()`).

8. Drawbacks / Things to Watch Out For

- Increases the number of classes in the codebase — one per strategy.
- Client/caller must be aware of the different strategies to choose the right one to inject.
- If strategies are very simple (a one-liner each), the overhead of separate classes may be unnecessary — a plain function/lambda reference might suffice in languages that support it.

9. Real-World Use Cases

- **Payment processing** — switching between credit card, debit card, UPI, wallet, etc. (this example).
- **Sorting algorithms** — plugging in different comparison/sorting strategies at runtime.
- **Compression algorithms** — choosing between ZIP, RAR, GZIP strategies for a file compressor.
- **Route calculation** — navigation apps switching between fastest route, shortest route, or avoid-tolls strategies.
- **Validation rules** — swapping different validation strategies for form/data validation depending on context.

10. Interview Questions & Answers

Q1. What is the Strategy Pattern? A behavioural design pattern that encapsulates a family of interchangeable algorithms into separate classes implementing a common interface, and lets a context object delegate to whichever one is currently set — allowing the algorithm to vary independently from the client that uses it.

Q2. What problem does it solve? It removes hardcoded, branching algorithm logic (`if-else` / `switch`) from a class, avoiding code duplication, maintenance headaches when switching

between algorithms, and violations of the Open/Closed Principle.

Q3. What are its main components/participants? - **Context** — holds a reference to a strategy and delegates work to it (`PaymentService`) - **Strategy** — interface defining the operation all algorithms must implement (`PaymentStrategy`) - **Concrete Strategy** — actual algorithm implementations (`CreditCard` , `DebitCard` , `UPI`)

Q4. How does the Strategy Pattern help follow the Open/Closed Principle? Because the context depends only on the `Strategy` interface, adding a new algorithm just means creating a new class that implements that interface — no existing code (including the context) needs to change.

Q5. How does Strategy differ from the State Pattern, since their structures look almost identical? Structurally they're very similar (a context holding an interface reference to interchangeable implementations), but their *intent* differs: Strategy is about choosing *how* to perform an algorithm/operation, typically set once by the client and not changed by the object itself. State is about representing an object's internal state transitions, where the object often changes its own "strategy" (state) as a result of its behavior.

Q6. What are the benefits? Open/Closed compliance, elimination of conditional complexity, single responsibility per algorithm, and the ability to swap behavior at runtime.

Q7. What are the drawbacks? More classes to maintain, and the client needs to know which concrete strategy to inject — plus it can be overkill for trivial algorithms.

Q8. Can you give real-world examples? Payment method selection, sorting/compression algorithm selection, navigation route strategies, and validation rule selection.

Q9. How does the client typically set a strategy on the context? Either by injecting it via a setter method (e.g. `setPaymentStrategy()` , used in this example) or by passing it into the context's constructor at creation time — both wire a concrete strategy object into the context before it's used.

11. Quick Recap

> **Problem:** Hardcoded, branching algorithm logic in one class makes it hard to maintain and violates the Open/Closed Principle. **Solution:** Encapsulate each algorithm in its own class implementing a common `Strategy` interface, and have the context delegate to whichever strategy is injected — enabling algorithms to be swapped freely at runtime.